Introduction to Computer Science
Fall 2026
Homework 2 Answer

Problem 1: Graph Basics

(a)

The connected components of G are $\{a, d, f\}$, $\{b, e\}$, and $\{c, g\}$. One cycle is $a \to d \to f \to a$.

For a graph with k connected components, the minimum number of new edges is $k - 1$. Each added edge decreases the number of components by at most one, so at least $k-1$ edges are necessary. Conversely, choose one vertex from each component and connect these representatives in a chain. This uses $k - 1$ edges and makes the graph connected.

For G, adding ab and ac connects its three components using two edges, which achieves the minimum.

(b)

Yes, T is a spanning tree of H: it contains all six vertices, is connected, and has no cycle. The unique simple path from c to f in T is

$$c \to b \to d \to e \to f.$$

Adding cf creates the cycle

$$c \to b \to d \to e \to f \to c.$$

To keep cf, we may remove any one of $bc,\ bd,\ de,\ ef$. Removing one of these edges breaks the cycle while preserving connectivity.

Problem 2: Algorithm and Time Complexity

(a)

(1) Merge Sort returns $[1, 3, 4, 6, 8, 9, 10]$. Median-of-Medians with $k = 4$ returns 6.

(2) For $n = 2^{30}$,

$$T_{\text{median}}(n) = 20 \cdot 2^{30}, \qquad T_{\text{sort}}(n) = 60 \cdot 2^{30}.$$

Median selection therefore performs one-third as many basic operations.

The two costs are equal when $20n = 2n \log_2 n$, giving $n = 2^{10} = 1024$.

(b)

For a binary heap, the total running time is $O((2n+m)\log n) = O((n+m)\log n)$. In the MST setting, we consider connected graphs, so $m \geq n-1$. Hence this bound simplifies to

$$O(m \log n).$$

For a Fibonacci heap, the total amortized running time is

$$O(n + m + n\log n) = O(m + n\log n).$$

(c)

Including the adjacency-list scans, Prim's algorithm with a Fibonacci heap has total amortized running time

$$O(m) + O(n) + O(n\log n) + O(m) = O(m + n\log n).$$

Yao's algorithm has a running-time bound of $O(m \log\log n)$. Thus:

Algorithm	$m = \Theta(n)$	$m = \Theta(n\log n)$
Prim with a Fibonacci heap	$O(n\log n)$	$O(n\log n)$
Yao's algorithm	$O(n\log\log n)$	$O(n\log n\log\log n)$

Yao's algorithm has the asymptotically smaller bound when $m = \Theta(n)$; Prim's algorithm with a Fibonacci heap has the asymptotically smaller bound when $m = \Theta(n\log n)$.

Problem 3: Shortest Path

(a)

Dijkstra's algorithm finalizes vertices in the order

$$S,\ B,\ A,\ D,\ C,\ E,\ F,\ T.$$

The shortest route is $S \to A \to C \to E \to T$, with total distance

$$400 + 300 + 300 + 300 = 1300 \text{ m}.$$

(b)

Using $f(v) = g(v) + h(v)$, A* finalizes vertices in the order

$$S,\ A,\ C,\ E,\ T.$$

Thus 5 vertices are finalized in total.

(c)

Let $h(v)$ be the supplied straight-line distance in metres. Use the time heuristic

$$h_t(v) = \frac{h(v)}{20000} \text{ hours.}$$

Any route from v to T has length at least $h(v)$, and Alice's speed never exceeds 20000 m/h. Its travel time is therefore at least $h(v)/20000$ hours, so h_t is admissible.

Using each road's travel time as its edge weight, A* finalizes vertices in the order

$$S,\ A,\ C,\ B,\ D,\ F,\ T.$$

The fastest route is $S \to B \to D \to F \to T$, with total travel time

$$\frac{200 + 400 + 500 + 600}{20000} = 0.085 \text{ hours.}$$